



Phase 4

1. The `this` Keyword

GOLDEN RULE: `this` ki value is baat par depend karti hai ki function ko **KAISE** call kiya gaya — na ki function **kahan likha** gaya hai. (Sirf arrow function iska exception hai.)

Matlab same function ko alag-alag tarike se call karoge toh `this` alag ho jaayega. Isiliye log confuse hote hain. Focus hamesha "**call-site**" par rakho — "function ko kis tarah bulaya gaya?"

1.1 — Global Scope mein `this` (Browser vs Node)

Browser mein top level par `this` poora `window` object hota hai:

```
console.log(this); // window (poora browser global object)
```

Node.js mein (kisi `.js` file ke andar) har file ek *module* hoti hai, isiliye top level par `this` ek khaali object hota hai:

```
console.log(this); // {} (yeh actually module.exports hai)
```

⚠ **Dhyaan do:** Node ke **REPL** (terminal par sirf `node` likhne par) mein `this` `global` hota hai, lekin `.js` file ke andar `this` `{}` (`module.exports`) hota hai. Dono jagah test karoge toh yeh difference dikhega.

1.2 — Regular Functions mein `this`

```
function show() {  
  console.log(this);  
}
```

```
}  
show();
```

- **Browser (non-strict):** `this` = `window`
- **Node (non-strict):** `this` = `global`
- **Strict mode mein:** `this` = `undefined`

Kyun? Jab function ko bina kisi object ke aise hi `show()` karke bulaate ho, JavaScript default global object de deta hai. Isko bolte hain **default binding**.

1.3 — Methods mein `this`

Jab function kisi object ke andar ho aur object ke through call ho:

```
const user = {  
  name: "Rahul",  
  greet() {  
    console.log("Hi, " + this.name);  
  }  
};  
user.greet(); // Hi, Rahul
```

💡 **Trick:** "Dot ke left mein dekho — jo wahan hai, wahi `this` hai." `user.greet()`
→ dot ke left `user` hai → `this = user`.

Ab catch yahan hai (important demo):

```
const fn = user.greet; // reference nikaal liya, object chhod  
diya  
fn(); // undefined (ab dot ke left koi object nahi)
```

Wahi function hai, lekin ab dot ke left koi object nahi — isliye `this` global ban gaya. **Yeh proof hai ki `this` "kaise call kiya" par depend karta hai, "kahan likha" par nahi.**

1.4 — Arrow Functions mein `this` (Lexical `this`) ★

Arrow function ka apna `this` **hota hi nahi**. Wo apne aas-paas (enclosing scope) se `this` udhaar le leta hai. Isko bolte hain **lexical this**.

```
const user = {
  name: "Rahul",
  greet: () => {
    console.log(this.name); // undefined! (user se nahi liya)
  }
};
user.greet();
```

Lekin yahi arrow function ek powerful jagah par kaam aata hai — jab andar **nested callback** ho:

```
const user = {
  name: "Rahul",
  hobbies: ["coding", "gaming"],
  show() {
    this.hobbies.forEach((hobby) => {
      // arrow ne 'this' ko show() se udhaar liya
      console.log(this.name + " likes " + hobby);
    });
  }
};
user.show();
// Rahul likes coding
// Rahul likes gaming
```

🔑 **Rule of thumb:** Method ko hamesha **regular function** banao (taaki `this` object ko point kare), aur uss method ke andar callbacks ke liye **arrow function** use karo (taaki `this` preserve rahe).

1.5 — Event Handlers mein `this`

Regular function handler mein `this` = wo HTML element jis par event laga:

```
button.addEventListener("click", function () {
  console.log(this); // wo button element jiske click hua
});
```

Arrow function handler mein `this` element ko point **nahi** karta — lexical scope (window) ko karta hai:

```
button.addEventListener("click", () => {
  console.log(this); // window (button NAHI)
});
```

💡 **Practical:** Handler ke andar clicked element chahiye toh **regular function** use karo, ya `event.target` use karo — jo dono cases mein kaam karta hai.

1.6 — Strict Mode mein `this`

`"use strict"` lagane se JavaScript thodi sakht ho jaati hai. Sabse bada change:

```
"use strict";
function show() {
  console.log(this); // undefined (non-strict mein window hot
a)
}
show();
```

Non-strict mein default binding global object deta tha; strict mode mein wo `undefined` de deta hai. Yeh accha hai — galti se global object ko ganda karne se bachata hai. (ES6 modules aur class bodies **automatically strict mode** mein hote hain.)

Quick Summary — `this` Cheatsheet

Kaise call kiya	<code>this</code> kya hoga
Global scope (browser)	<code>window</code>
Global scope (Node module)	<code>{}</code> (module.exports)

Kaise call kiya	<code>this</code> kya hoga
Regular function <code>fn()</code>	global object (strict mein <code>undefined</code>)
Method <code>obj.fn()</code>	<code>obj</code> (dot ke left wala)
Arrow function	parent / lexical scope ka <code>this</code>
Event handler (regular fn)	event wala element
Event handler (arrow)	lexical <code>this</code> (window)

2. `call`, `apply`, `bind` — Explicit Binding

Kabhi-kabhi hum khud decide karna chahte hain ki `this` kya hoga. Iske liye 3 methods hain — inhe bolte hain **explicit binding**.

```
function introduce(city, country) {
  console.log(`Main ${this.name} hoon, ${city}, ${country} se`);
}
const person = { name: "Rahul" };
```

`call` — turant call karo, arguments comma se

```
introduce.call(person, "Indore", "India");
// Main Rahul hoon, Indore, India se
```

Pehla argument = `this` ki value. Baaki arguments normal comma-separated.

`apply` — turant call karo, arguments ARRAY mein

```
introduce.apply(person, ["Indore", "India"]);
// Main Rahul hoon, Indore, India se
```

💡 **Yaad rakhne ka tarika:** Apply = Array | Call = Comma. (A se A, C se C.)

`bind` — call MAT karo, naya function bana ke do

```
const boundFn = introduce.bind(person);
boundFn("Indore", "India");
// Main Rahul hoon, Indore, India se
```

`bind` function ko call nahi karta — wo ek **naya function return** karta hai jiska `this` hamesha ke liye `person` par fix (lock) ho jaata hai. Baad mein jab marzi call karo.

🔗 **Real use:** Event handlers aur callbacks mein jahan `this` lost ho raha ho, wahan `bind` se permanently fix kar dete hain. Jaise

```
setTimeout(introduce.bind(person), 1000).
```

Method	Kab call hota hai	Arguments
<code>call</code>	Turant	Comma-separated
<code>apply</code>	Turant	Ek array mein
<code>bind</code>	Baad mein (naya fn deta hai)	Comma-separated

3. Prototypes

3.1 — Prototype kya hai?

Har JavaScript object ke andar ek **chhupa hua link** hota hai jo kisi **doosre object** ko point karta hai. Uss doosre object ko bolte hain uska **prototype**.

Fayda: agar object mein koi property/method nahi mila, toh JS uske prototype mein dhoondhega; wahan bhi nahi mila toh prototype ke prototype mein... aur aise hi upar tak. Isi chain ko bolte hain **prototype chain**. Socho jaise — bachche ko cheez nahi mili toh papa se maangega, papa ke paas nahi toh dada se.

3.2 — `__proto__` vs `prototype` (Sabse confusing pair)

	<code>__proto__</code>	<code>prototype</code>
Kis par hota hai?	Har object par	Sirf functions (constructor) par
Kya hai?	Object ke prototype tak ka actual link	Wo object jo <code>new</code> se bane instances ka <code>__proto__</code> banega

	<code>__proto__</code>	<code>prototype</code>
Kaam	"Mera prototype ye hai" (lookup ke liye)	"Mere instances ka prototype ye hoga" (blueprint)

Code se proof:

```
function Person(name) {
  this.name = name;
}
const p = new Person("Rahul");

console.log(p.__proto__ === Person.prototype); // true
```

🔗 **Ek line mein:** `prototype` = function ki property (instances ke liye blueprint).
`__proto__` = instance ki property (uske blueprint tak ka raasta). Connection:
`instance.__proto__ === Function.prototype`.

⚠️ **Note:** `__proto__` aaj "legacy" maana jaata hai (samajhne ke liye best).
 Production mein `Object.getPrototypeOf(obj)` use karte hain.

3.3 — Prototype Chain

```
const arr = [1, 2, 3];
arr.push(4); // push kahan se aaya? arr ke andar toh likha hi nahi!
```

JS ne aise dhoondha:

- `arr` mein → nahi mila
- `arr.__proto__` (jo `Array.prototype` hai) mein → **mil gaya!**

```
arr → Array.prototype → Object.prototype → null
```

Jab `null` aa gaya aur cheez nahi mili → `undefined` return hota hai. Yeh hai prototype chain.

3.4 — `Object.create`

Yeh method directly ek object banata hai jiska prototype tum khud set karte ho:

```
const animal = {
  eats: true,
  walk() { console.log("Animal chal raha hai"); }
};

const dog = Object.create(animal); // dog ka prototype = animal
dog.barks = true;

console.log(dog.barks); // true (apna)
console.log(dog.eats); // true (animal se inherited!)
dog.walk();             // Animal chal raha hai
```

3.5 — Inheritance via Prototypes

```
const vehicle = {
  start() { console.log(this.name + " start ho gayi"); }
};

const car = Object.create(vehicle);
car.name = "Swift";
car.start(); // Swift start ho gayi
```

`car` ne `start` method `vehicle` se inherit kiya, aur jab method chala toh `this` = `car` raha (dot ke left wala rule!). Isliye `this.name` = "Swift". **Yahi prototypal inheritance ka core idea hai.**

4. ES6 Classes

🔴 **Pehle samajh lo:** Class ke saath `new` keyword aata hai. Jab `new MyClass()` chalta hai toh JS 4 cheezein karta hai: (1) ek naya khaali object banta hai, (2)

uska `__proto__` class ke prototype par set hota hai, (3) constructor chalta hai jisme `this` = wahi naya object, (4) wo object automatically return ho jaata hai.

4.1 — `class` Syntax + `constructor` + Instance Methods

```
class Person {  
  // new chalne par yahi automatically call hota hai  
  constructor(name, age) {  
    this.name = name;  
    this.age = age;  
  }  
  // instance method → automatically prototype par jaata hai  
  greet() {  
    console.log("Hi, main " + this.name);  
  }  
}  
  
const p = new Person("Rahul", 25);  
p.greet(); // Hi, main Rahul
```

- `constructor` → setup ka kaam (properties assign).
- `greet()` jaise methods → **automatically** `Person.prototype` par chale jaate hain (har object mein copy nahi banti — memory bachti hai).

4.2 — Inheritance: `extends` & `super`

```
class Animal {  
  constructor(name) { this.name = name; }  
  eat() { console.log(this.name + " kha raha hai"); }  
}  
  
class Dog extends Animal {  
  constructor(name, breed) {  
    super(name); // parent ka constructor call  
    this.breed = breed;  
  }  
}
```

```

    }
    bark() { console.log(this.name + " bhonk raha hai"); }
}

const d = new Dog("Tommy", "Labrador");
d.eat(); // Tommy kha raha hai (Animal se inherited)
d.bark(); // Tommy bhonk raha hai (Dog ka apna)

```

- `extends` → ek class ko doosri se inherit karwata hai.
- `super(name)` → parent ke constructor ko call karta hai.

⚠ **Rule:** Child constructor mein `this` use karne se **pehle** `super()` call karna zaroori hai, warna error. `super.methodName()` se parent ka method bhi call kar sakte ho.

4.3 — Static Methods & Static Properties

`static` wali cheezein **object par nahi, class par** hoti hain. Inhe instance se nahi, **directly class se** call karte ho.

```

class MathHelper {
    static PI = 3.14159; // static property
    static add(a, b) { return a + b; } // static method
}

console.log(MathHelper.add(2, 3)); // 5 (class se call)
console.log(MathHelper.PI); // 3.14159

const m = new MathHelper();
// m.add(2, 3); → Error: static method instance par nahi hota

```

📌 **Real example:** `Array.isArray()` ek static method hai. Usko `{}.isArray()` nahi, balki `Array.isArray([])` aise call karte ho. Utility/helper functions ke liye static perfect hai.

4.4 — Getters & Setters

In se hum method ko property ki tarah use karte hain (bina `()` lagaye).

```
class Person {
  constructor(first, last) {
    this.first = first;
    this.last = last;
  }
  get fullName() { // getter
    return this.first + " " + this.last;
  }
  set fullName(value) { // setter
    const parts = value.split(" ");
    this.first = parts[0];
    this.last = parts[1];
  }
}

const p = new Person("Rahul", "Sharma");
console.log(p.fullName); // Rahul Sharma (bina () ke!)
p.fullName = "Priya Verma"; // setter chala
console.log(p.first); // Priya
```

Dhyaan do — `p.fullName` likha, `p.fullName()` nahi. Getter/setter property jaisa feel dete hain lekin andar logic chalta hai. Validation ke liye bahut useful (setter mein check laga sakte ho).

4.5 — Private Fields (`#`)

`#` lagane se field **bahar se access nahi** ho sakta — sirf class ke andar.

```
class BankAccount {
  #balance = 0; // private field

  deposit(amount) { this.#balance += amount; }
  getBalance() { return this.#balance; }
}
```

```
const acc = new BankAccount();
acc.deposit(500);
console.log(acc.getBalance()); // 500
// console.log(acc.#balance); → SyntaxError: bahar se access
nahi!
```

`#balance` ko sirf class ke andar wale methods chhoo sakte hain. Yeh **encapsulation** deta hai — important data ko galat changes se bachata hai.

4.6 — Big Reveal: Classes sirf Syntactic Sugar hain ★★

"Class" JavaScript mein koi nayi cheez nahi hai. Andar-andar yeh wahi purana **prototype + constructor function** hai, bas dikhne mein sundar. Isko bolte hain **"syntactic sugar"** — meethi packaging, andar wahi cheez.

Proof 1 — Class actually ek function hi hai:

```
class Person {
  constructor(name) { this.name = name; }
  greet() { return "Hi " + this.name; }
}
console.log(typeof Person); // "function" (class nahi, function!)
```

Proof 2 — Methods prototype par hi gaye hain:

```
console.log(typeof Person.prototype.greet); // "function"
// matlab greet method Person.prototype par pada hai — purane
tareeke jaise
```

Proof 3 — Instance ka link wahi hai:

```
const p = new Person("Rahul");
console.log(p.__proto__ === Person.prototype); // true
```

Proof 4 — Bilkul same cheez purane tareeke se:

```
// Yeh class...
class Person {
  constructor(name) { this.name = name; }
  greet() { return "Hi " + this.name; }
}

// ...exactly is constructor function ke barabar hai:
function Person(name) {
  this.name = name;
}
Person.prototype.greet = function () {
  return "Hi " + this.name;
};
```

Dono ka behaviour 100% same. `class` sirf likhne ka aasaan aur saaf tareeka hai — andar JavaScript prototype hi use kar rahi hai.



Yaad rakho: Agli baar koi `class` dekho, toh yaad rakhna — peeche chhupa hua wahi prototype hai jo shuru mein padha tha. JavaScript ne use sirf sundar kapde pehna diye hain.